On Hold · presenter run sheet

Wednesday 28 October 2026 · 10–15 minutes · slides: On-Hold-Presentation.pdf (or the .html: arrows to move, F for full screen)

Timing

WHEN	SLIDES	WHAT TO GET ACROSS
0:00	1–5	R2.4 billion lost to digital banking crime in 2025; scams win in the moment before someone checks. On Hold is for older South Africans and their families: one question, a plain verdict, family told only by consent.
3:00	6 · Demo video	Play the video (about 7 minutes). Slide 6 lists what it shows; talk over it or let it run.
10:00	7–14	Architecture, the AI as the core feature, the accuracy test, trust and security, CRUD, process, feedback, what's next. About 25 seconds a slide.
13:30	15	Questions. Offer to open the code (see the table on the next page).

Before you record, and before you present

1. Before the final take, fresh demo data and Marie's and Pieter's hourly AI limit reset: `cd backend` then `uv run python -m scripts.seed`
2. Check the live API: `on-hold-ashen.vercel.app/health` should show `claude-haiku-5-5`.
3. Fill in the peer feedback on slide 13 (three rows), then re-export the PDF.
4. The scam screenshot for demo step 3 is ready: `presentation/demo-scam-sms.png` (a fake SASSA SMS; the live app gives it a Stop).
5. Allow the microphone for onhold.pages.dev and try Speak once.
6. For recording: onhold.pages.dev, and the app signed in as admin in a second browser profile. Browser zoom about 125% so the phone reads well on video. Notifications off.
7. For presenting: the slides, the demo video file (its YouTube link as a backup), the GitHub repo.
8. Editor open on `backend/app/ai.py`, `backend/app/main.py`, `backend/scripts/eval.py`, `mobile/src/screens/CheckResult.tsx`.

Recording script for the demo video (about 7 minutes)

1. **Website** · 30 s
   Open onhold.pages.dev. "This is deployed: the website and app on Cloudflare Pages, the API on Vercel, data on Supabase." Tap **Try the demo** in the phone.

2. **Marie checks a WhatsApp** · 1½ min
   Tap the **Marie** chip → Sign in. Home → **Check something**, paste:

   > **Hi Ma, it's me. My phone fell in the water so this is my new number. Can you send R2,500 for a supplier urgently? I'll pay you back tomorrow. Don't tell Dad.**

   Check it. Point out the breathing pause (on purpose, at least 3 seconds). **Stop** → open **Why we think this** → **Official advice** (a vetted source) → **How this scam takes your money** (the dark "what they're counting on" card) → back → **Tell family**.

3. **A screenshot** · 45 s
   Check → **Screenshot** → pick `presentation/demo-scam-sms.png` → Check it → Stop, citing SASSA.

4. **A call in progress** · 1 min
   Home → **Someone on the phone right now?** → **Speak** and say:

   > **A man says he's from my bank's fraud department and he wants the code they just sent me.**

   Fix any words, Check it → **Say this, then hang up**. (Mic fails? Type it.)

5. **A warning poster** · 30 s
   Check → paste:

   > **WARNING FROM THE RTIA: Beware of fake AARTO traffic fine SMSes offering a discount for paying today. They are scams. Never click links in fine messages.**

   → grey **Scam warning**: "a real warning is never a green tick, and a fake one is a Stop."

6 **Report it, the map, Learn** · 1 min
Home → History → open the Stop from step 2 → **Report it** → submit ("it waits for a moderator"). **Map**: heat by suburb, filter by type. **Learn**: what's going around, the guides.

7 **Pieter, the guardian** · 30 s
Tap the circle with Marie's initial (top right of Home) → **Sign out** → **Pieter** chip → **Needs you** shows Marie's Stop and her Tell-family request.

8 **Admin** · 1½ min
Switch to the admin tab (admin@onhold.test, your private password) → **Admin panel**: **Dashboard** (counts per verdict, the 14-day trend) → **Activity** (Marie's check and report from a minute ago) → **Reports**: approve Marie's report, now on the map → **People** → Marie: history, family, activity.

> **If a take goes wrong:** reseed for fresh data and record that step again. A check takes about 4 seconds; the first after a quiet spell can take 5–6. On the day, if the video file won't play, open its YouTube link.

Likely questions, and what to open

QUESTION	SHOW
Where is the AI, and why is it core?	`backend/app/ai.py`: `SYSTEM_PROMPT`, `ScamVerdict`, `run_check`. Every check goes through it; voice goes through Whisper first (`transcribe`).
How do you stop prompt injection?	The `<message>` tags and the rule in `SYSTEM_PROMPT`: an instruction inside a message is itself a scam signal.
How do you know it's accurate?	`backend/scripts/eval.py`: 29 labelled messages, 29/29 on Sonnet 5 and Haiku 5.5, no dangerous misses. Results in `docs/testing.md`.
Can the AI invent a link?	No: `SOURCES` and `SourceId` in `ai.py`; the schema only allows those 8 ids.
Why Haiku and not a bigger model?	Same score on the test, about 16× cheaper (R0.005 a check), a bit faster. `/health` reports the live model.
How is data secured?	`backend/app/auth.py` (Supabase JWT checked against its keys); queries scoped to the user in `main.py` (e.g. `get_check`); RLS on with no policies in `supabase/migrations`.
Rate limiting and cost?	`_check_rate_limit` in `main.py`: 30 AI calls an hour per person, with an advisory lock. Prompt caching in `run_check`.
How does family consent work?	`create_check` in `main.py`: a Stop alerts only links with `share_verdicts` or `share_calls` on.
Show the CRUD.	Reports and admin routes in `main.py`; all 48 listed in `backend/docs/api.md`.
Deployment?	Vercel: `backend/vercel.json` (Frankfurt) and the transaction pooler (`backend/app/db.py`). Cloudflare Pages: `website/README.md`.
Version control and process?	GitHub network graph: feature → dev → main with merge commits; CI in `.github/workflows/ci.yml`.
What went wrong along the way?	The bug log in `docs/testing.md` (42 entries), e.g. the live API hanging until it moved to the transaction pooler.